LAB IMPLEMENTATION EXPERIMENT 6

Experiment 6: Variational Autoencoders (VAEs) for Image Generation

Generative AI Laboratory (CO3)

Date: 22 October 2026

Aim

To implement a convolutional Variational Autoencoder (VAE) for image generation, and to use it for reconstruction, sampling, latent-space interpolation and latent-space visualisation.

Theory

A **Variational Autoencoder** (Kingma and Welling, 2014) is a probabilistic generative model with a latent variable z. An *encoder* network gives an approximate posterior $q_\phi(z|x) = \mathcal{N}(\mu_\phi(x), \sigma^2_\phi(x))$ and a *decoder* network models $p_\theta(x|z)$. Training maximises the evidence lower bound (ELBO)

$$\log p_\theta(x) \;\geq\; \mathbb{E}_{q_\phi(z|x)}[\log p_\theta(x|z)] - \mathrm{KL}\big(q_\phi(z|x) \,\|\, p(z)\big), \qquad p(z) = \mathcal{N}(0, I).$$

The first term is the reconstruction quality; the KL term (closed form for Gaussians, $-\frac{1}{2}\sum(1 + \log\sigma^2 - \mu^2 - \sigma^2)$) keeps the latent space close to a standard normal, so *new images can be generated by sampling $z \sim \mathcal{N}(0, I)$ and decoding it*. Gradients flow through the random sampling thanks to the **reparameterisation trick** $z = \mu + \sigma \odot \epsilon$. Weighting the KL term by β (β-VAE) trades reconstruction sharpness against a smoother, more disentangled latent space. Compared to GANs, VAEs train stably and give an explicit latent space, but their samples are usually blurrier.

Steps

1. Load MNIST and scale the pixels to $[0, 1]$.
2. Build the encoder (two strided convolutions, dense layer, μ and $\log\sigma^2$ heads), the sampling layer and the transposed-convolution decoder.
3. Implement the ELBO loss in a custom `train_step` and train a 16-dimensional VAE for 15 epochs.
4. Plot the loss components, reconstruct unseen test digits, sample new digits from the prior and interpolate between two real digits.
5. Train a 2-D VAE and plot the encoded test digits and the decoded latent manifold.
6. Train VAEs with $\beta \in \{0.1, 1, 4\}$ and compare the reconstruction/KL trade-off.

Code

```
import os
```

```
os.environ["TF_CPP_MIN_LOG_LEVEL"] = "3"
import numpy as np, matplotlib.pyplot as plt
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
keras.utils.set_random_seed(3)
(x_train, y_train), (x_test, y_test) = keras.datasets.mnist.load_data()
x_train = (x_train / 255.0).astype("float32")[..., None]; x_test = (x_test / 255.0).astype("float32")[..., None]
print("train", x_train.shape, "test", x_test.shape, "| pixel range [0,1] (Bernoulli decoder)")

class Sampling(layers.Layer):
    def call(self, inputs):
        mu, logvar = inputs
        return mu + tf.exp(0.5 * logvar) * tf.random.normal(tf.shape(mu))

def build_encoder(latent):
    inp = keras.Input((28, 28, 1))
    x = layers.Conv2D(32, 3, strides=2, padding="same", activation="relu")(inp)
    x = layers.Conv2D(64, 3, strides=2, padding="same", activation="relu")(x)
    x = layers.Flatten()(x); x = layers.Dense(128, activation="relu")(x)
    mu, logvar = layers.Dense(latent, name="mu")(x), layers.Dense(latent, name="logvar")(x)
    return keras.Model(inp, [mu, logvar, Sampling()([mu, logvar])], name="encoder")

def build_decoder(latent):
    inp = keras.Input((latent,))
    x = layers.Dense(7 * 7 * 64, activation="relu")(inp); x = layers.Reshape((7, 7, 64))(x)
    x = layers.Conv2DTranspose(64, 3, strides=2, padding="same", activation="relu")(x)
    x = layers.Conv2DTranspose(32, 3, strides=2, padding="same", activation="relu")(x)
    return keras.Model(inp, layers.Conv2DTranspose(1, 3, padding="same", activation="sigmoid")(x), name="decoder")

class VAE(keras.Model):
    def __init__(self, latent, beta=1.0):
        super().__init__()
        self.enc, self.dec, self.beta = build_encoder(latent), build_decoder(latent), beta
        self.trk = {k: keras.metrics.Mean(name=k) for k in ("loss", "recon", "kl")}
    @property
    def metrics(self): return list(self.trk.values())
    def train_step(self, x):
        with tf.GradientTape() as tape:
            mu, logvar, z = self.enc(x, training=True)
            rec = self.dec(z, training=True)
            recon = tf.reduce_mean(tf.reduce_sum(keras.losses.binary_crossentropy(x, rec), axis=(1, 2)))
            kl = tf.reduce_mean(-0.5 * tf.reduce_sum(1 + logvar - tf.square(mu) - tf.exp(logvar), axis=1))
            loss = recon + self.beta * kl
        self.optimizer.apply_gradients(zip(tape.gradient(loss, self.trainable_weights), self.trainable_weights))
        for k, v in zip(("loss", "recon", "kl"), (loss, recon, kl)): self.trk[k].update_state(v)
```

```
        return {k: m.result() for k, m in self.trk.items()}

vae16 = VAE(16)
vae16.enc.summary(print_fn=print); vae16.dec.summary(print_fn=print)

vae16.compile(optimizer=keras.optimizers.Adam(1e-3))
h16 = vae16.fit(x_train, epochs=15, batch_size=128, verbose=2)

fig, ax = plt.subplots(1, 3, figsize=(11, 3))
for a, k, t in zip(ax, ("loss", "recon", "kl"), ("Total loss (-ELBO)", "
    Reconstruction (BCE)", "KL divergence")):
    a.plot(range(1, 16), h16.history[k], "o-"); a.set_title(t); a.set_xlabel("epoch
    ")
plt.tight_layout(); plt.show()

mu, _, _ = vae16.enc(x_test[:12], training=False)
rec = vae16.dec(mu).numpy()
fig, axes = plt.subplots(2, 12, figsize=(13, 2.6))
for i in range(12):
    axes[0, i].imshow(x_test[i, ..., 0], cmap="gray"); axes[1, i].imshow(rec[i, ...,
     0], cmap="gray")
    axes[0, i].axis("off"); axes[1, i].axis("off")
axes[0, 0].set_title("input", fontsize=8, loc="left"); axes[1, 0].set_title("
    reconstruction", fontsize=8, loc="left")
plt.tight_layout(); plt.show()
mse = float(np.mean((vae16.dec(vae16.enc(x_test[:2000])[0]).numpy() - x_test
    [:2000]) ** 2))
print("Test reconstruction MSE (2000 images): %.5f" % mse)

z = tf.random.normal((100, 16), seed=4)
gen = vae16.dec(z).numpy()[..., 0]
grid = np.concatenate([np.concatenate(gen[r * 10:(r + 1) * 10], axis=1) for r in
    range(10)], axis=0)
plt.figure(figsize=(6, 6)); plt.imshow(grid, cmap="gray"); plt.axis("off"); plt.
    title("100 generated digits (latent dim 16)"); plt.show()

def interp(i, j, steps=10):
    za, zb = vae16.enc(x_test[[i]])[0], vae16.enc(x_test[[j]])[0]
    zs = tf.concat([(1 - a) * za + a * zb for a in np.linspace(0, 1, steps)], 0)
    return vae16.dec(zs).numpy()[..., 0]
pairs = [(0, 1), (2, 3), (4, 5)]
fig, axes = plt.subplots(3, 10, figsize=(12, 3.8))
for r, (i, j) in enumerate(pairs):
    for c, im in enumerate(interp(i, j)):
        axes[r, c].imshow(im, cmap="gray"); axes[r, c].axis("off")
    axes[r, 0].set_title(f"digit {y_test[i]} -> {y_test[j]}", fontsize=8, loc="left
    ")
plt.tight_layout(); plt.show()

vae2 = VAE(2); vae2.compile(optimizer=keras.optimizers.Adam(1e-3))
h2 = vae2.fit(x_train, epochs=15, batch_size=128, verbose=2)
print("final loss (2-D latent): %.2f  vs  (16-D latent): %.2f" % (h2.history["loss"
    ][-1], h16.history["loss"][-1]))

mu2 = vae2.enc.predict(x_test[:6000], verbose=0)[0]
fig, ax = plt.subplots(1, 2, figsize=(11, 5))
sc = ax[0].scatter(mu2[:, 0], mu2[:, 1], c=y_test[:6000], cmap="tab10", s=3)
```

```
ax[0].legend(*sc.legend_elements(), title="digit", fontsize=7, loc="upper right"); ax[0].set_title("Encoded test digits (mean mu)")
n = 15; xs = np.linspace(-3, 3, n)
canvas = np.zeros((28 * n, 28 * n))
for i, yv in enumerate(xs[::-1]):
    dec = vae2.dec(np.array([[xv, yv] for xv in xs], dtype="float32")).numpy()[..., 0]
    for j in range(n): canvas[i * 28:(i + 1) * 28, j * 28:(j + 1) * 28] = dec[j]
ax[1].imshow(canvas, cmap="gray", extent=[-3, 3, -3, 3]); ax[1].set_title("Decoded 15x15 manifold of z in [-3,3]^2")
ax[1].set_xlabel("z1"); ax[1].set_ylabel("z2")
plt.tight_layout(); plt.show()

res = {}
for beta in (0.1, 1.0, 4.0):
    m = VAE(16, beta); m.compile(optimizer=keras.optimizers.Adam(1e-3))
    hh = m.fit(x_train, epochs=6, batch_size=128, verbose=0)
    res[beta] = (hh.history["recon"][-1], hh.history["kl"][-1], m)
    print(f"beta={beta:<4}  reconstruction loss {res[beta][0]:6.2f}   KL {res[beta][1]:6.2f}")
fig, axes = plt.subplots(2, 3, figsize=(8, 5.4))
zz = tf.random.normal((16, 16), seed=9)
for c, (beta, (_, _, m)) in enumerate(res.items()):
    g = m.dec(zz).numpy()[..., 0]
    axes[0, c].imshow(np.concatenate([np.concatenate(g[r * 4:(r + 1) * 4], 1) for r in range(4)], 0), cmap="gray")
    axes[0, c].set_title(f"samples, beta={beta}", fontsize=9); axes[0, c].axis("off")
    rr = m.dec(m.enc(x_test[:8])[0]).numpy()[..., 0]
    axes[1, c].imshow(np.concatenate([np.concatenate(rr[:4], 1), np.concatenate(rr[4:], 1)], 0), cmap="gray")
    axes[1, c].set_title("reconstructions", fontsize=9); axes[1, c].axis("off")
plt.tight_layout(); plt.show()
```

Listing 1. `experiment_code.py`

Output

Imports and data

```
train (60000, 28, 28, 1) test (10000, 28, 28, 1) | pixel range [0,1] (Bernoulli decoder)
```

Defining the VAE (encoder, sampling layer, decoder, training step)

```
Model: "encoder"
+---------------------+-------------------+------------+--------------------+
| Layer (type)        | Output Shape      |    Param # | Connected to       |
+---------------------+-------------------+------------+--------------------+
| input_layer         | (None, 28, 28, 1) |          0 | -                  |
| (InputLayer)        |                   |            |                    |
+---------------------+-------------------+------------+--------------------+
| conv2d (Conv2D)     | (None, 14, 14,    |        320 | input_layer[0][0]  |
|                     | 32)               |            |                    |
+---------------------+-------------------+------------+--------------------+
| conv2d_1 (Conv2D)   | (None, 7, 7, 64)  |     18,496 | conv2d[0][0]       |
```

```
+---------------------+-------------------+------------+-------------------+
| flatten (Flatten)   | (None, 3136)      |          0 | conv2d_1[0][0]    |
+---------------------+-------------------+------------+-------------------+
| dense (Dense)       | (None, 128)       |    401,536 | flatten[0][0]     |
+---------------------+-------------------+------------+-------------------+
| mu (Dense)          | (None, 16)        |      2,064 | dense[0][0]       |
+---------------------+-------------------+------------+-------------------+
| logvar (Dense)      | (None, 16)        |      2,064 | dense[0][0]       |
+---------------------+-------------------+------------+-------------------+
| sampling (Sampling) | (None, 16)        |          0 | mu[0][0],         |
|                     |                   |            | logvar[0][0]      |
+---------------------+-------------------+------------+-------------------+
 Total params: 424,480 (1.62 MB)
 Trainable params: 424,480 (1.62 MB)
 Non-trainable params: 0 (0.00 B)

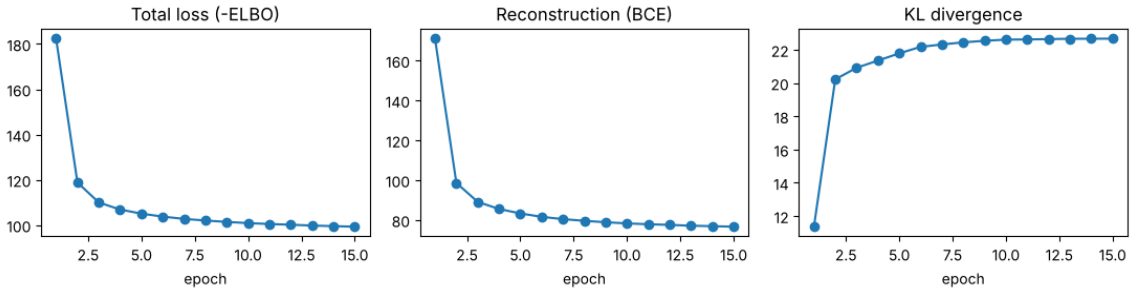
Model: "decoder"
+---------------------------------+------------------------+---------------+
| Layer (type)                    | Output Shape           |       Param # |
+---------------------------------+------------------------+---------------+
| input_layer_1 (InputLayer)      | (None, 16)             |             0 |
+---------------------------------+------------------------+---------------+
| dense_1 (Dense)                 | (None, 3136)           |        53,312 |
+---------------------------------+------------------------+---------------+
| reshape (Reshape)               | (None, 7, 7, 64)       |             0 |
+---------------------------------+------------------------+---------------+
| conv2d_transpose                | (None, 14, 14, 64)     |        36,928 |
| (Conv2DTranspose)               |                        |               |
+---------------------------------+------------------------+---------------+
| conv2d_transpose_1              | (None, 28, 28, 32)     |        18,464 |
| (Conv2DTranspose)               |                        |               |
+---------------------------------+------------------------+---------------+
| conv2d_transpose_2              | (None, 28, 28, 1)      |           289 |
| (Conv2DTranspose)               |                        |               |
+---------------------------------+------------------------+---------------+
 Total params: 108,993 (425.75 KB)
 Trainable params: 108,993 (425.75 KB)
 Non-trainable params: 0 (0.00 B)
```

Training the 16-dimensional VAE

```
Epoch 1/15
469/469 - 38s - 81ms/step - kl: 11.3589 - loss: 182.3420 - recon: 170.9830
Epoch 2/15
469/469 - 38s - 81ms/step - kl: 20.2216 - loss: 118.8186 - recon: 98.5968
Epoch 3/15
469/469 - 41s - 87ms/step - kl: 20.9167 - loss: 110.0143 - recon: 89.0976
Epoch 4/15
469/469 - 34s - 72ms/step - kl: 21.3510 - loss: 106.8629 - recon: 85.5118
Epoch 5/15
469/469 - 33s - 70ms/step - kl: 21.7837 - loss: 105.0649 - recon: 83.2812
Epoch 6/15
469/469 - 41s - 86ms/step - kl: 22.1750 - loss: 103.7471 - recon: 81.5721
Epoch 7/15
469/469 - 32s - 68ms/step - kl: 22.3196 - loss: 102.7399 - recon: 80.4202
Epoch 8/15
469/469 - 33s - 71ms/step - kl: 22.4465 - loss: 102.0263 - recon: 79.5798
Epoch 9/15
469/469 - 40s - 86ms/step - kl: 22.5401 - loss: 101.4087 - recon: 78.8687
Epoch 10/15
469/469 - 33s - 70ms/step - kl: 22.6105 - loss: 100.8870 - recon: 78.2765
Epoch 11/15
469/469 - 32s - 68ms/step - kl: 22.6215 - loss: 100.4843 - recon: 77.8628
Epoch 12/15
```

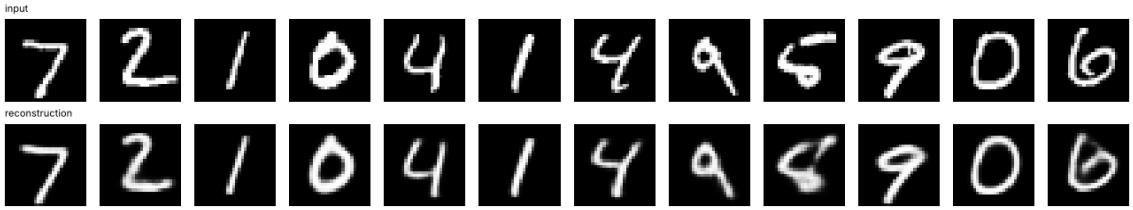
```
469/469 - 32s - 69ms/step - kl: 22.6425 - loss: 100.1741 - recon: 77.5316
Epoch 13/15
469/469 - 32s - 69ms/step - kl: 22.6616 - loss: 99.7928 - recon: 77.1311
Epoch 14/15
469/469 - 33s - 70ms/step - kl: 22.6635 - loss: 99.5455 - recon: 76.8821
Epoch 15/15
469/469 - 32s - 68ms/step - kl: 22.6684 - loss: 99.3107 - recon: 76.6423
```

Loss curves



Reconstructions of unseen test digits

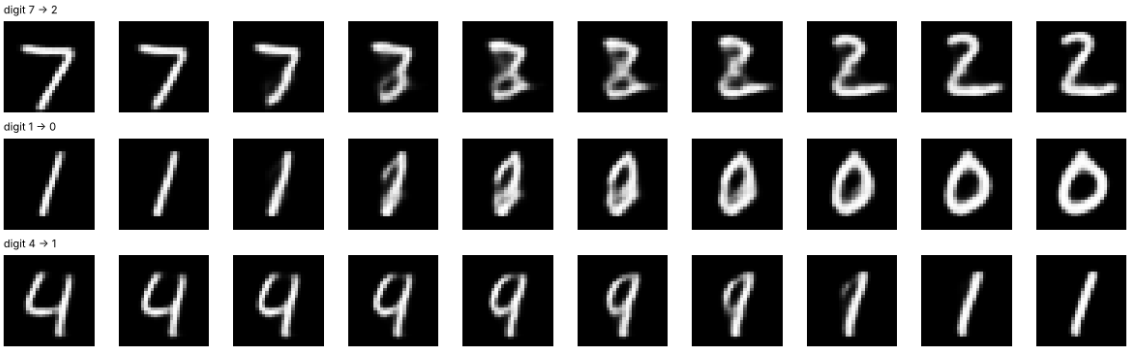
```
Test reconstruction MSE (2000 images): 0.00922
```



New digits sampled from the prior z ~ N(0, I)



Interpolation between two real digits in latent space

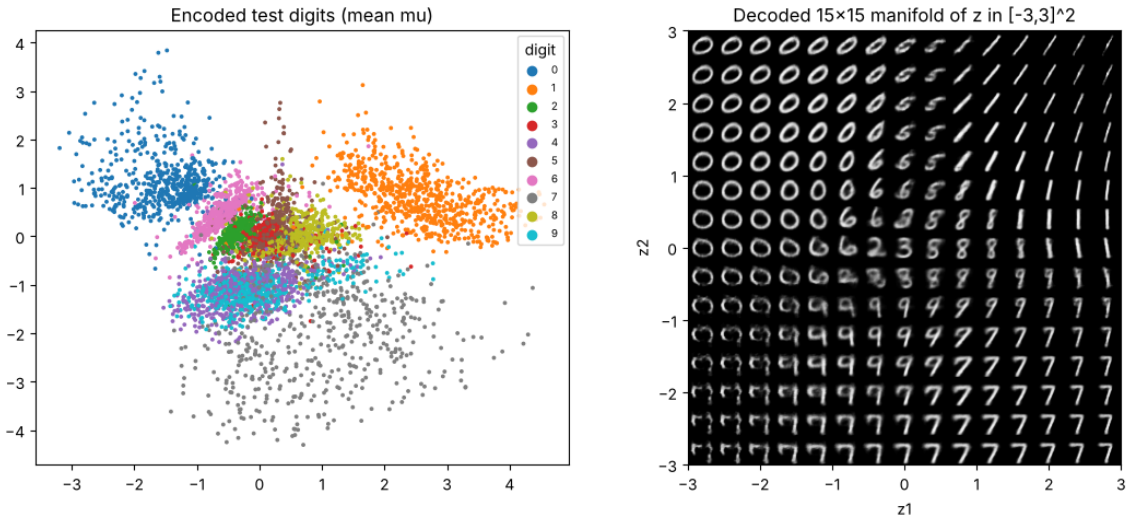


Training a 2-dimensional VAE

```
Epoch 1/15
469/469 - 34s - 72ms/step - kl: 3.9947 - loss: 200.9043 - recon: 196.9096
Epoch 2/15
469/469 - 32s - 68ms/step - kl: 4.9596 - loss: 166.0773 - recon: 161.1178
Epoch 3/15
469/469 - 37s - 78ms/step - kl: 5.5102 - loss: 157.8461 - recon: 152.3359
Epoch 4/15
469/469 - 30s - 64ms/step - kl: 5.7894 - loss: 154.3515 - recon: 148.5621
Epoch 5/15
469/469 - 27s - 57ms/step - kl: 5.9414 - loss: 152.2564 - recon: 146.3149
Epoch 6/15
469/469 - 47s - 99ms/step - kl: 6.0580 - loss: 150.6968 - recon: 144.6388
Epoch 7/15
469/469 - 33s - 69ms/step - kl: 6.1472 - loss: 149.5249 - recon: 143.3777
```

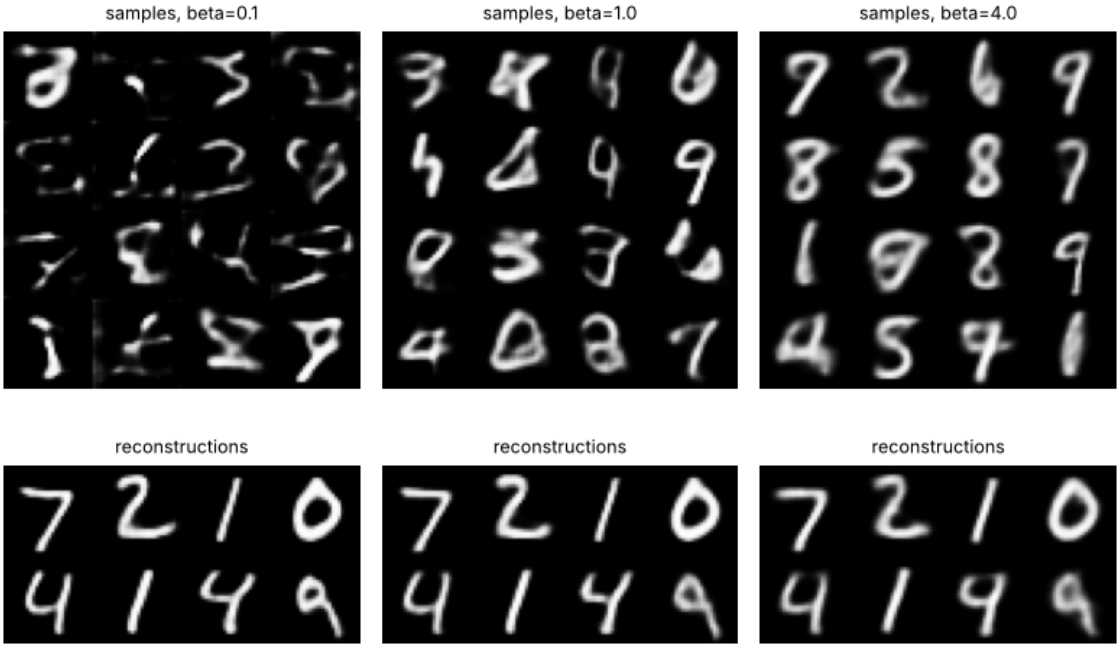
```
Epoch 8/15
469/469 - 33s - 70ms/step - kl: 6.2127 - loss: 148.6507 - recon: 142.4380
Epoch 9/15
469/469 - 33s - 70ms/step - kl: 6.2701 - loss: 147.8423 - recon: 141.5722
Epoch 10/15
469/469 - 32s - 69ms/step - kl: 6.3345 - loss: 147.2110 - recon: 140.8766
Epoch 11/15
469/469 - 33s - 70ms/step - kl: 6.3646 - loss: 146.6782 - recon: 140.3135
Epoch 12/15
469/469 - 31s - 66ms/step - kl: 6.3970 - loss: 146.1459 - recon: 139.7490
Epoch 13/15
469/469 - 42s - 90ms/step - kl: 6.4464 - loss: 145.7013 - recon: 139.2548
Epoch 14/15
469/469 - 22s - 48ms/step - kl: 6.4891 - loss: 145.3401 - recon: 138.8509
Epoch 15/15
469/469 - 17s - 36ms/step - kl: 6.5035 - loss: 144.9120 - recon: 138.4085
final loss (2-D latent): 144.91  vs  (16-D latent): 99.31
```

Latent space of the 2-D VAE: encoded test digits and decoded manifold



Reconstruction vs KL trade-off for beta in {0.1, 1, 4}

```
beta=0.1   reconstruction loss  68.76   KL  48.16
beta=1.0   reconstruction loss  82.37   KL  21.47
beta=4.0   reconstruction loss 111.18   KL   9.10
```



Conclusion

The VAE learned a smooth, structured latent space of handwritten digits. After 15 epochs the 16-dimensional model reached a negative ELBO of 99.3 nats (reconstruction 76.6 + KL 22.7) and a test reconstruction MSE of 0.0092; reconstructions of unseen digits were faithful, digits sampled from the prior were recognisable (but blurrier than GAN samples), and interpolations changed gradually from one digit to another. The 2-D VAE had a higher loss (144.9) because of its tiny bottleneck, but it arranged the digit classes in clusters whose decoded manifold shows continuous morphing between digits. Varying β showed the expected trade-off: from $\beta = 0.1$ to $\beta = 4$ the reconstruction loss rose from 68.8 to 111.2 while the KL term fell from 48.2 to 9.1, i.e. a stronger KL weight gives a more regular latent space at the cost of blurrier reconstructions.